

Removing Deprecated Exception Specifications from C++17

Table of Contents

- [Introduction](#)
- [A Brief History of Exception Specifications](#)
- [Proposed Changes](#)
- [Motivation for Change](#)
- [Compatibility Concerns](#)
- [Proposed Wording](#)
 - [3.3.7 Class scope \[basic.scope.class\]](#)
 - [3.4.1 Unqualified name lookup \[basic.lookup.unqual\]](#)
 - [8.4.2 Explicitly-defaulted functions \[dcl.fct.def.default\]](#)
 - [9.2 Class members \[class.mem\]](#)
 - [14.5 Template declarations \[temp.decls\]](#)
 - [14.5.3 Variadic templates \[temp.variadic\]](#)
 - [14.6 Name resolution \[temp.res\]](#)
 - [14.6.4.1 Point of instantiation \[temp.point\]](#)
 - [14.7.1 Implicit instantiation \[temp.inst\]](#)
 - [14.7.2 Explicit instantiation \[temp.explicit\]](#)
 - [14.8.2 Template argument deduction \[temp.deduct\]](#)
 - [15.4 Exception specifications \[except.spec\]](#)
 - [15.5 Special functions \[except.special\]](#)
 - [15.5.1 The `std::terminate\(\)` function \[except.terminate\]](#)
 - [15.5.2 The `std::unexpected\(\)` function \[except.unexpected\]](#)
 - [17.6.5.12 Restrictions on exception handling \[res.on.exception.handling\]](#)
 - [18.8 Exception Handling \[support.exception\]](#)
 - [Header `<exception>` synopsis](#)
 - [18.8.2 Class `bad\_exception` \[bad.exception\]](#)
 - [23.3.2.8 Zero sized arrays \[array.zero\]](#)
 - [Annex B \(informative\) Implementation quantities \[implimits\]](#)
 - [D.4 Dynamic exception specifications \[depr.except.spec\]](#)
 - [D.8 Violating *exception-specifications* \[exception.unexpected\]](#)
 - [D.8.1 Type `unexpected\_handler` \[unexpected.handler\]](#)
 - [D.8.2 `set\_unexpected` \[set.unexpected\]](#)
 - [D.8.3 `get\_unexpected` \[get.unexpected\]](#)
 - [D.8.4 `unexpected` \[unexpected\]](#)
- [Acknowledgements](#)
- [References](#)

Introduction

Dynamic exception specifications were deprecated in C++11. This paper formally proposes removing the feature from C++17, while retaining the (still) deprecated `throw()` specification strictly as an alias for `noexcept(true)`.

A Brief History of Exception Specifications

Exception specifications were added as part of the original design of the exception language feature. However, the experience of using them was less than desired, with the general consensus by the time the 1998 standard was published being, generally, to not use them. The standard library made the explicit choice to not use exception specifications apart from a handful of places where guaranteeing an empty (no-throwing) specification seemed useful. [N741](#) gives a brief summary of the LWG thoughts at the time.

By the time C++11 was published, the feeling against exception specifications had grown, the limited real-world use generally

reported negative user experience, and the language feature, renamed as *dynamic* exception specifications, was deprecated, (N3051). A new language feature, `noexcept`, was introduced to describe the important case of knowing when a function could guarantee to not throw any exceptions.

Looking ahead to C++17, there is a desire to incorporate exception specifications into the type system, N4533. This solves a number of awkward corners that arise from exception specifications not being part of the type of a function, but still does nothing for the case of deprecated dynamic exception specifications, so the actual language does not get simpler, and we must still document and teach the awkward corners that arise from dynamic exception specifications outside the type system.

Proposed Changes

The recommendation of this paper is to remove dynamic exception specifications from the language. However, the syntax of the `throw()` specification should be retained, but no longer as a dynamic exception specification. Its meaning should become strictly an alias for `noexcept(true)`, and its usage should remain deprecated.

To minimize the impact on the current standard wording, the grammar term *exception-specification* is retained, although it has only one production and could be replaced entirely with *noexcept-specification*. Alternatively, the grammar term *noexcept-specification* could be retired.

The wording changes in this initial proposal are deliberately minimal in an effort to ensure the least risk of accidental change. However, rather than using the language of sets of compatible exception specifications (where there are now only two such sets, the empty set and the set of all types) it would be possible to write a simpler, clearer form describing instead functions that permit exceptions, and functions that permit *no* exceptions. While such a specification would be preferred, it is also beyond the drafting skills of the proposal author.

The redrafting goes slightly beyond minimal by eliminating use of the pseudo-type "any". This change, while improving clarity, also avoids confusion with the standard library type `any` from the Library Fundamentals TS in the event that it becomes part of a future standard library.

Motivation for Change

Dynamic exception specifications are a failed experiment, but this is not immediately clear to novices, especially where the "novice" is an experienced developer coming from other languages such as Java, where exception specifications may be more widely used. Their continuing presence, especially in an important part of the language model (exceptions are key to understanding constructors, destructors, and RAII) is an impediment that must be explained. It remains embarrassing to explain to new developers that there are features of the language that conforming compilers are required to support, yet should never be used.

Exception specifications in general occupy an awkward corner of the grammar, where they do not affect the type system, yet critically affect how a virtual function can be overridden, or which functions can bind to certain function pointer variables. As noted above, N4533 would go a long way to resolving that problem for `noexcept` exception specifications, which makes the hole left for dynamic exception specifications even more awkward and unusual for the next generation of C++ developers.

C++17 is on schedule to be a release with several breaking changes for old code, with the standard library removing `auto_ptr`, the classic C++98 function binders, and more. Similarly, it is expected that the core language will lose trigraphs, the `register` keyword, and the increment operator for `bool`. This would be a good time to make a clean break with long discouraged (and actively deprecated) parts of the language.

The proposed change would resolve core issue 596 as no longer relevant (NAD), and should simplify core issue 1351, although that is marked as a Defect Report that was not yet applied to the working paper the proposed wording below was drafted from.

Compatibility Concerns

There is certainly some body of existing code that has not heeded the existing best practice of discouraging dynamic exception specifications, and has not yet accounted for the feature being deprecated in C++11. It is not clear how much of such code would be expected to port unmodified into a C++17 (or beyond) world, and the change is relatively simple - just strike the (non-empty) dynamic exception specification to retain equivalent meaning. The key difference is that the `unexpected` handler will not now be called to translate unexpected exceptions. In rare cases, this would allow a new exception type to propagate, rather than calling `terminate` to abort. If enforcing that semantic is seen as important in production systems, there is a more intrusive workaround available:

```
void legacy() throw(something)
try
```

```

{
    // function body as before
}
catch(const something&) {
    throw;
}
catch(...) {
    terminate();
}

```

It is thought that the empty dynamic exception specification, `throw()`, was much more widely used in practice, often in the mistaken impression that compilers would use this information to optimize code generation where no exception could propagate, where in fact this is strictly a pessimization that forces stack unwinding to the function exit, and then calling the unexpected callback in a manner that is guaranteed to fail before the subsequent call to `terminate`. This paper proposes treating such (still deprecate) exception specifications as synonyms for `noexcept(true)`, yielding the performance benefit many were originally expecting.

It should also be noted that at least one widely distributed compiler has still not implemented this feature in 2015, and at least one vendor has expressed a desire to never implement the deprecated feature (while that vendor *has* implemented the `noexcept` form of exception specification). Code on that platform would not be adversely impacted by the proposed removal, and portable code must always have allowed for the idiosyncracies of this platform.

One remaining task is to survey popular open source libraries and see what level of usage, if any, remains in large, easily accessible codebases.

Proposed Wording

3.3.7 Class scope [basic.scope.class]

1 The following rules describe the scope of names declared in classes.

1) The potential scope of a name declared in a class consists not only of the declarative region following the name’s point of declaration, but also of all function bodies, default arguments, *exception-specifications*, and *brace-or-equal-initializers* of non-static data members in that class (including such things in nested classes).

3.4.1 Unqualified name lookup [basic.lookup.unqual]

7 A name used in the definition of a class *x* outside of a member function body, default argument, *exception-specification*, *brace-or-equal-initializer* of a non-static data member, or nested class definition²⁹ shall be declared in one of the following ways:

8 For the members of a class *x*, a name used in a member function body, in a default argument, *in-an-exception-specification*, in the *brace-or-equal-initializer* of a non-static data member (9.2), or in the definition of a class member outside of the definition of *x*, following the member’s *declarator-id*₃₁, shall be declared in one of the following ways:

31) That is, an unqualified name that occurs, for instance, in a type in the *parameter-declaration-clause* *or in the exception-specification*.

8.4.2 Explicitly-defaulted functions [dcl.fct.def.default]

4 [*Example*:

```

struct S {
    constexpr S() = default;           // ill-formed: implicit S() is not constexpr
    S(int a = 0) = default;           // ill-formed: default argument
    void operator=(const S&) = default; // ill-formed: non-matching return type
    ~S() noexcept(false) throw(int) = default; // deleted: exception specification does not match
private:
    int i;
    S(S&);                             // OK: private copy constructor
};
S::S(S&) = default;                   // OK: defines copy constructor

```

— *end example*]

9.2 Class members [class.mem]

2 A class is considered a completely-defined object type (3.9) (or complete type) at the closing `}` of the *class-specifier*. Within the

class *member-specification*, the class is regarded as complete within function bodies, default arguments, *using-declarations* introducing inheriting constructors (12.9), ~~*exception-specifications*~~; and *brace-or-equal-initializers* for non-static data members (including such things in nested classes). Otherwise it is regarded as incomplete within its own class *member-specification*.

3 [*Note*: A single name can denote several function members provided their types are sufficiently different (Clause13). — *end note*]

4 A *brace-or-equal-initializer* shall appear only in the declaration of a data member. (For static data members, see 9.4.2; for non-static data members, see 12.6.2). A *brace-or-equal-initializer* for a non-static data member shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class ~~or the exception specification of that constructor~~.

14.5 Template declarations [temp.decls]

2 For purposes of name lookup and instantiation, default arguments ~~and exception-specifications~~ of function templates and default arguments ~~and exception-specifications~~ of member functions of class templates are considered definitions; each default argument ~~or exception-specification~~ is a separate definition which is unrelated to the function template definition or to any other default arguments ~~or exception-specifications~~.

14.5.3 Variadic templates [temp.variadic]

~~(4.7) — In a dynamic-exception-specification (15.4); the pattern is a type-id.~~

14.6 Name resolution [temp.res]

11 [*Note*: For purposes of name lookup, default arguments ~~and exception-specifications~~ of function templates and default arguments ~~and exception-specifications~~ of member functions of class templates are considered definitions(14.5). — *end note*]

14.6.4.1 Point of instantiation [temp.point]

~~3 For an exception-specification of a function template specialization or specialization of a member function of a class template, if the exception-specification is implicitly instantiated because it is needed by another template specialization and the context that requires it depends on a template parameter, the point of instantiation of the exception-specification is the point of instantiation of the specialization that requires it. Otherwise, the point of instantiation for such an exception-specification immediately follows the namespace scope declaration or definition that requires the exception-specification.~~

14.7.1 Implicit instantiation [temp.inst]

1 Unless a class template specialization has been explicitly instantiated (14.7.2) or explicitly specialized (14.7.3), the class template specialization is implicitly instantiated when the specialization is referenced in a context that requires a completely-defined object type or when the completeness of the class type affects the semantics of the program. [*Note*: Within a template declaration, a local class or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, ~~exception-specifications~~, and non-static data member initializers, if any). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated as part of the instantiation of the entity within which the local class or enumeration is declared. — *end note*] The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not of the definitions, ~~or~~ default arguments, ~~or exception-specifications~~ of the class member functions, member classes, scoped member enumerations, static data members and member templates; and it causes the implicit instantiation of the definitions of unscoped member enumerations and member anonymous unions. However, for the purpose of determining whether an instantiated redeclaration of a member is valid according to 9.2, a declaration that corresponds to a definition in the template is considered to be a definition. [*Example*:

~~15 The exception-specification of a function template specialization is not instantiated along with the function declaration; it is instantiated when needed (15.4). If such an exception-specification is needed but has not yet been instantiated, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the exception-specification is done as if it were being done as part of instantiating the declaration of the specialization at that point.~~

14.7.2 Explicit instantiation [temp.explicit]

12 The usual access checking rules do not apply to names used to specify explicit instantiations. [*Note*: In particular, the template arguments and names used in the function declarator (including parameter types; ~~and~~ return types ~~and exception-specifications~~) may be private types or objects which would normally not be accessible and the template may be a member template or member function which would not normally be accessible. — *end note*]

14.8.2 Template argument deduction [temp.deduct]

7 The substitution occurs in all types and expressions that are used in the function type and in template parameter declarations. The expressions include not only constant expressions such as those that appear in array bounds or as nontype template arguments but also general expressions (i.e., non-constant expressions) inside `sizeof`, `decltype`, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered. [Note: The equivalent substitution in exception specifications is done only when the exception specification is instantiated, at which point a program is ill-formed if the substitution results in an invalid type or expression. — end note]

15.4 Exception specifications [except.spec]

1 The *exception specification* of a function is a (possibly empty) set of types, indicating that the function might exit via an exception ~~that matches a handler of one of the types in the set~~; the (conceptual) set of all types is used to denote that the function might exit via an exception of arbitrary type. If the set is empty, the function is said to have a *non-throwing exception specification*. The exception specification is either defined explicitly by using an *exception-specification* as a suffix of a function declaration's declarator (8.3.5) or implicitly.

```
exception-specification:  
  dynamic-exception-specification  
  noexcept-specification  
  noexcept ( constant-expression )  
  noexcept  
  throw ( )
```

```
dynamic-exception-specification:  
  throw ( type-id-listopt )
```

```
type-id-list:  
  type-id ...opt  
  type-id-list , type-id ...opt
```

```
noexcept-specification:  
  noexcept ( constant-expression )  
  noexcept
```

In an exception~~noexcept~~-specification, the *constant-expression*, if supplied, shall be a constant expression (5.20) that is contextually converted to `bool` (Clause 4). A `(` token that follows `noexcept` is part of the exception~~noexcept~~-specification and does not commence an initializer (8.5). The exception-specification `throw()` is deprecated (see Annex D), and equivalent to the exception-specification `noexcept`.

2 An *exception-specification* shall appear only on a function declarator for a function type, pointer to function type, reference to function type, or pointer to member function type that is the top-level type of a declaration or definition, or on such a type appearing as a parameter or return type in a function declarator. An *exception-specification* shall not appear in a `typedef` declaration or *alias-declaration*. [Example:

```
void f() noexceptthrow(int);           // OK  
void (*fp)() noexceptthrow(int);       // OK  
void g(void pfa() noexceptthrow(int)); // OK  
typedef int (*pf)() noexceptthrow(int); // ill-formed
```

— end example]

~~A type denoted in a dynamic-exception-specification shall not denote an incomplete type or an rvalue reference type. A type denoted in a dynamic-exception-specification shall not denote a pointer or reference to an incomplete type, other than "pointer to cv void". A type cv T, "array of T", or "function returning T" denoted in a dynamic-exception-specification is adjusted to type T, "pointer to T", or "pointer to function returning T", respectively. A dynamic-exception-specification denotes an exception specification that is the set of adjusted types specified thereby.~~

3 The *exception-specification* `noexcept` OR `noexcept (constant-expression)`, where the *constant-expression* yields `true`, denotes an exception specification that is the empty set. The *exception-specification* `noexcept (constant-expression)`, where the *constant-expression* yields `false`, or the absence of an *exception-specification* in a function declarator other than that for a destructor (12.4) or a deallocation function (3.7.4.2) denotes an exception specification that is the set of all types.

4 Two *exception-specifications* are *compatible* if the sets of types they denote are the same.

5 If any declaration of a function has an *exception-specification* ~~that is not a noexcept-specification~~ allowing no~~all~~ exceptions, all declarations, including the definition and any explicit specialization, of that function shall have a compatible *exception-specification*. If any declaration of a pointer to function, reference to function, or pointer to member function has an *exception-specification*, all occurrences of that declaration shall have a compatible *exception-specification*. If a declaration of a function has an implicit exception specification, other declarations of the function shall not specify an *exception-specification*. In an explicit instantiation an *exception-specification* may be specified, but is not required. If an *exception-specification* is specified in an

explicit instantiation directive, it shall be compatible with the *exception-specifications* of other declarations of that function. A diagnostic is required only if the *exception-specifications* are not compatible within a single translation unit.

6 If a virtual function has an exception specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall only allow exceptions that are allowed by the exception specification of the base class virtual function, unless the overriding function is defined as deleted. [*Example:*

```
struct B {
    virtual void f() noexcept throw (int, double);
    virtual void g();
};

struct D: B {
    void f();                // ill-formed
    void g() noexcept throw (int);    // OK
};
```

The declaration of `D::f` is ill-formed because it allows all exceptions, whereas `B::f` allows no exceptions~~only int and double~~. — *end example*] A similar restriction applies to assignment to and initialization of pointers to functions, pointers to member functions, and references to functions: the target entity shall allow at least the exceptions allowed by the source value in the assignment or initialization. [*Example:*

```
class A { /âˆ—...âˆ—/ };
void (*pf1)();    // no exception specification
void (*pf2)() noexcept throw (A);

void f() {
    pf1 = pf2;    // OK: pf1 is less restrictive
    pf2 = pf1;    // error: pf2 is more restrictive
}
```

— *end example*]

7 In such an assignment or initialization, *exception-specifications* on return types and parameter types shall be compatible. In other assignments or initializations, *exception-specifications* shall be compatible.

~~8 An exception-specification can include the same type more than once and can include classes that are related by inheritance, even though doing so is redundant. [Note: An exception-specification can also include the class `std::bad_exception` (18.8.2). â€” end note]~~

~~9 A function is said to allow an exception of type E if its exception specification contains a type T for which a handler of type T would be a match (15.3) for an exception of type E . A function is said to allow all exceptions if its exception specification is the set of all types. Otherwise a function does not allow any exceptions.~~

10 Whenever an exception ~~of type E~~ is thrown and the search for a handler (15.3) encounters the outermost block of a function with an exception specification that does not allow any exceptions E , then, the function `std::terminate()` is called (15.5.1).

~~(10.1) — if the function definition has a dynamic-exception-specification, the function `std::unexpected()` is called (15.5.2);~~

~~(10.2) — otherwise, the function `std::terminate()` is called (15.5.1).~~

[*Example:*

```
class X {};
class Y {};
class Z: public X {};
class W {};

void f() throw (X, Y) {
    int n = 0;
    if (n) throw X(); // OK
    if (n) throw Z(); // also OK
    throw W(); // will call std::unexpected()
}
```

— *end example*]

~~[Note: A function can have multiple declarations with different non-throwing exception-specifications; for this purpose, the one on the function definition is used. — end note]~~

11 An implementation shall not reject an expression merely because when executed it throws or might throw an exception that the containing function does not allow. [*Example:*

```
extern void f() throw(X,Y);
void g() throw(X) {
    f();    // OK
}
```

the call to `f` is well-formed even though when called, `f` might throw an exception ~~`Y`~~ that `g` does not allow. — *end example*]

12 [*Note*: An exception specification is not considered part of a function's type; see 8.3.5. — *end note*]

13 A potential exception of a given context is either a type that might be thrown as an exception or a pseudo-type, denoted by "any", that represents the situation where an exception of an arbitrary type might be thrown. A subexpression `e1` of an expression `e` is an immediate subexpression if there is no subexpression `e2` of `e` such that `e1` is a subexpression of `e2`.

14 The set of potential exceptions of a function, function pointer, or member function pointer `f` is the set denoted by the exception specification of `f`, defined as follows:

(14.1) — ~~If the exception specification of `f` is the set of all types, the set consists of the pseudo-type "any".~~

(14.2) — ~~Otherwise, the set consists of every type in the exception specification of `f`.~~

15 The set of potential exceptions of an expression `e` is empty if `e` is a core constant expression (5.20). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of `e`, including default argument expressions used in a function call, combined with a set `s` defined by the form of `e`, as follows:

(15.1) — If `e` is a function call (5.2.2):

(15.1.1) — If its *postfix-expression* is a (possibly parenthesized) *id-expression* (5.1.1), class member access (5.2.5), or pointer-to-member operation (5.5) whose *cast-expression* is an *id-expression*, `s` is the set of potential exceptions of the entity selected by the contained *id-expression* (after overload resolution, if applicable).

(15.1.2) — Otherwise, `s` is the set of all types~~contains the pseudo-type "any".~~

(15.2) — If `e` implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if `e` is a full-expression (1.9)), `s` is the set of potential exceptions of the function.

(15.3) — if `e` is a throw-expression (5.17), `s` is the set of all types~~consists of the type of the exception object that would be initialized by the operand, if present, or the pseudo-type "any" otherwise.~~

(15.4) — if `e` is a `dynamic_cast` expression that casts to a reference type and requires a run-time check (5.2.7), `s` is the set of all types~~consists of the type `std::bad_cast`.~~

(15.5) — if `e` is a `typeid` expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8), `s` is the set of all types~~consists of the type `std::bad_typeid`.~~

(15.6) — if `e` is a *new-expression* with a non-constant expression in the *noptr-new-declarator* (5.3.4), `s` is the set of all types~~consists of the type `std::bad_array_new_length`.~~

[*Example*: Given the following declarations

```
void f() throw(int)noexcept(false);
void g();
struct A { A(); };
struct B { B() noexcept; };
struct D { D() throw(double)noexcept(false); };
```

the set of potential exceptions for some sample expressions is:

(15.~~76.1~~) — for `f()`, the set is the set of all types ~~consists of `int`~~;

(15.~~86.2~~) — for `g()`, the set is the set of all types ~~consists of "any"~~;

(15.~~96.3~~) — for `new A`, the set is the set of all types ~~consists of "any"~~;

(15.~~106.4~~) — for `B()`, the set is empty;

(15.~~116.5~~) — for `new D`, the set is the set of all types ~~consists of "any" and `double`.~~

— *end example*]

16 Given a member function `f` of some class `x`, where `f` is an inheriting constructor (12.9) or an implicitly-declared special member function, the *set of potential exceptions of the implicitly-declared member function `f`* consists of all the members from the

following sets:

(16.1) — if f is a constructor,

(16.1.1) — the sets of potential exceptions of the constructor invocations

(16.1.1.1) — for x 's non-variant non-static data members,

(16.1.1.2) — for x 's direct base classes, and

(16.1.1.3) — if x is non-abstract (10.4), for x 's virtual base classes,

(including default argument expressions used in such invocations) as selected by overload resolution for the implicit definition of f (12.1). [*Note:* Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (15.2), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor could never escape the constructor (15.1, 15.5.1). — *end note*]

(16.1.2) — the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* that are not ignored (12.6.2);

(16.2) — if f is an assignment operator, the sets of potential exceptions of the assignment operator invocations for x 's non-variant non-static data members and for x 's direct base classes (including default argument expressions used in such invocations), as selected by overload resolution for the implicit definition of f (12.8);

(16.3) — if f is a destructor, the sets of potential exceptions of the destructor invocations for x 's non-variant non-static data members and for x 's virtual and direct base classes.

17 An inheriting constructor (12.9) and an implicitly-declared special member function (Clause 12) are considered to have an implicit exception specification, as follows, where s is the set of potential exceptions of the implicitly-declared member function:

(17.1) — if s is the set of all types~~contains the pseudo-type "any"~~, the implicit exception specification is the set of all types;

(17.2) — otherwise, the implicit exception specification contains all the types in s .

[*Note:* An instantiation of an inheriting constructor template has an implied exception specification as if it were a non-template inheriting constructor. — *end note*] [*Example:*

```
struct A {
    A(int = (A(5), 0)) noexcept;
    A(const A&) throw();
    A(A&&) throw();
    ~A() throw(X);;
};
struct B {
    B() throw();
    B(const B&) = default; // exception specification contains no types
    B(B&&, int = (throw Y(), 0)) noexcept;
    ~B() noexcept(false)throw(Y);
};
int n = 7;
struct D : public A, public B {
    int * p = new (std::nothrow) int[n];
    // exception specification of D::D() contains the set of all typesXand std::bad_array_new_length
    // exception specification of D::D(const D&) contains no types
    // exception specification of D::D(D&&) contains the set of all typesY
    // exception specification of D::~D() contains the set of all typesXand Y
};
```

Furthermore, if $A::\sim A()$ ~~or~~ $B::\sim B()$ were virtual, $D::\sim D()$ would not be as restrictive as that of $A::\sim A$, and the program would be ill-formed since a function that overrides a virtual function from a base class shall have an *exception-specification* at least as restrictive as that in the base class. — *end example*]

18 A deallocation function (3.7.4.2) with no explicit *exception-specification* has an exception specification that is the empty set.

19 An *exception-specification* is considered to be *needed* when:

(19.1) — in an expression, the function is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4);

(19.2) — the function is odr-used (3.2) or, if it appears in an unevaluated operand, would be odr-used if the expression were potentially-evaluated;

(19.3) — the *exception-specification* is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);

(19.4) — the function is defined; or

(19.5) — the *exception-specification* is needed for a defaulted special member function that calls the function. [*Note:* A defaulted declaration does not require the *exception-specification* of a base member function to be evaluated until the implicit *exception-specification* of the derived function is needed, but an explicit *exception-specification* needs the implicit *exception-specification* to compare against. — *end note*]

The *exception-specification* of a defaulted special member function is evaluated as described above only when needed; similarly, the *exception-specification* of a specialization of a function template or member function of a class template is instantiated only when needed.

~~20 In a dynamic-exception-specification, a type-id followed by an ellipsis is a pack expansion (14.5.3).~~

~~21 [*Note:* The use of dynamic-exception-specifications is deprecated (see Annex D). — *end note*]~~

15.5 Special functions [except.special]

1 The functions `std::terminate()` (15.5.1) ~~and `std::unexpected()` (15.5.2) are~~is used by the exception handling mechanism for coping with errors related to the exception handling mechanism itself. The function `std::current_exception()` (18.8.5) and the class `std::nested_exception` (18.8.6) can be used by a program to capture the currently handled exception.

15.5.1 The `std::terminate()` function [except.terminate]

1 In some situations exception handling must be abandoned for less subtle error handling techniques. [*Note:* These situations are:

(1.1) — when the exception handling mechanism, after completing the initialization of the exception object but before activation of a handler for the exception (15.1), calls a function that exits via an exception, or

(1.2) — when the exception handling mechanism cannot find a handler for a thrown exception (15.3), or

(1.3) — when the search for a handler (15.3) encounters the outermost block of a function with ~~an *exception*~~*noexcept-specification* that does not allow the exception (15.4), or

(1.4) — when the destruction of an object during stack unwinding (15.2) terminates by throwing an exception, or

(1.5) — when initialization of a non-local variable with static or thread storage duration (3.6.2) exits via an exception, or

(1.6) — when destruction of an object with static or thread storage duration exits via an exception (3.6.3), or

(1.7) — when execution of a function registered with `std::atexit` or `std::at_quick_exit` exits via an exception (18.5), or

(1.8) — when a *throw-expression* (5.17) with no operand attempts to rethrow an exception and no exception is being handled (15.1), or

~~(1.9) — when `std::unexpected` exits via an exception of a type that is not allowed by the previously violated exception specification, and `std::bad_exception` is not included in that exception specification (15.5.2), or~~

~~(1.10) — when the implementation[™]s default unexpected exception handler is called (D.8.1), or~~

(1.11) — when the function `std::nested_exception::rethrow_nested` is called for an object that has captured no exception (18.8.6), or

(1.12) — when execution of the initial function of a thread exits via an exception (30.3.1.2), or

(1.13) — when the destructor or the copy assignment operator is invoked on an object of type `std::thread` that refers to a joinable thread (30.3.1.3, 30.3.1.4), or

(1.14) — when a call to a `wait()`, `wait_until()`, or `wait_for()` function on a condition variable (30.5.1, 30.5.2) fails to meet a postcondition.

— *end note*]

2 In such cases, `std::terminate()` is called (18.8.3). In the situation where no matching handler is found, it is implementation-defined whether or not the stack is unwound before `std::terminate()` is called. In the situation where the search for a handler

(15.3) encounters the outermost block of a function with an *exception*~~noexcept~~-specification that does not allow the exception (15.4), it is implementation-defined whether the stack is unwound, unwound partially, or not unwound at all before `std::terminate()` is called. In all other situations, the stack shall not be unwound before `std::terminate()` is called. An implementation is not permitted to finish stack unwinding prematurely based on a determination that the unwind process will eventually cause a call to `std::terminate()`.

15.5.2 The `std::unexpected()` function [except.unexpected]

1 If a function with a *dynamic-exception-specification* exits via an exception of a type that is not allowed by its exception specification, the function `std::unexpected()` is called (D.8) immediately after completing the stack unwinding for the former function.

2 [Note: By default, `std::unexpected()` calls `std::terminate()`, but a program can install its own handler function (D.8.2). In either case, the constraints in the following paragraph apply. — end note]

3 The `std::unexpected()` function shall not return, but it can throw (or rethrow) an exception. If it throws a new exception which is allowed by the exception specification which previously was violated, then the search for another handler will continue at the call of the function whose exception specification was violated. If it exits via an exception of a type that the *dynamic-exception-specification* does not allow, then the following happens: If the *dynamic-exception-specification* does not include the class `std::bad_exception` (18.8.2) then the function `std::terminate()` is called, otherwise the thrown exception is replaced by an implementation-defined object of type `std::bad_exception` and the search for another handler will continue at the call of the function whose *dynamic-exception-specification* was violated.

4 [Note: Thus, a *dynamic-exception-specification* guarantees that a function exits only via an exception of one of the listed types. If the *dynamic-exception-specification* includes the type `std::bad_exception` then any exception type not on the list may be replaced by `std::bad_exception` within the function `std::unexpected()`. — end note]

17.6.5.12 Restrictions on exception handling [res.on.exception.handling]

1 Any of the functions defined in the C++ standard library can report a failure by throwing an exception of a type described in its **Throws**: paragraph. An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing *exception*~~noexcept~~-specification.

2 A function may throw an object of a type not listed in its **Throws** clause if its type is derived from a type named in the **Throws** clause and would be caught by an exception handler for the base type.

3 Functions from the C standard library shall not throw exceptions¹⁹¹ except when such a function calls a program-supplied function that throws an exception.¹⁹²

4 Destructor operations defined in the C++ standard library shall not throw exceptions. Every destructor in the C++ standard library shall behave as if it had a non-throwing exception specification. Any other functions defined in the C++ standard library that do not have an *exception-specification* may throw implementation-defined exceptions unless otherwise specified.¹⁹³ An implementation may strengthen this implicit *exception-specification* by adding an explicit one.¹⁹⁴

¹⁹⁴ That is, an implementation may provide an explicit *exception-specification* that defines the subset of "any" exceptions thrown by that function. This implies that the implementation may list implementation-defined types in such an *exception-specification*.

18.8 Exception Handling [support.exception]

1 The header `<exception>` defines several types and functions related to the handling of exceptions in a C++ program.

Header `<exception>` synopsis

```
namespace std {
    class exception;
    class bad_exception;
    class nested_exception;

    typedef void (*unexpected_handler)();
    unexpected_handler get_unexpected() noexcept;
    unexpected_handler set_unexpected(unexpected_handler f) noexcept;
    [[noreturn]] void unexpected();

    typedef void (*terminate_handler)();
    terminate_handler get_terminate() noexcept;
    terminate_handler set_terminate(terminate_handler f) noexcept;
```

```

[[noreturn]] void terminate() noexcept;

int uncaught_exceptions() noexcept;
// D.9X, uncaught_exception (deprecated)
bool uncaught_exception() noexcept;

typedef unspecified exception_ptr;

exception_ptr current_exception() noexcept;
[[noreturn]] void rethrow_exception(exception_ptr p);
template <class E> exception_ptr make_exception_ptr(E e) noexcept;
template <class T> [[noreturn]] void throw_with_nested(T&& t);
template <class E> void rethrow_if_nested(const E& e);
}

```

18.8.2 Class bad_exception [bad.exception]

1 The class bad_exception defines the type of objects referenced by the exception_ptr returned from a call to current_exception (18.8.5 [propagation]) when the currently active exception object fails to copy~~thrown as described in (15.5.2).~~

23.3.2.8 Zero sized arrays [array.zero]

4 Member function swap() shall have an exception~~noexcept~~-specification which is equivalent to noexcept(true).

Annex B (informative) Implementation quantities [implimits]

~~(2.40) — Throw specifications on a single function declaration [256].~~

D.4 Dynamic exception specifications [depr.except.spec]

1 The exception-specification ~~throw()~~use of dynamic exception specifications is deprecated.

~~D.8 Violating exception specifications [exception.unexpected]~~

~~D.8.1 Type unexpected_handler [unexpected.handler]~~

~~typedef void (*unexpected_handler)();~~

~~1 The type of a handler function to be called by unexpected() when a function attempts to throw an exception not listed in its dynamic exception specification.~~

~~2 Required behavior: An unexpected_handler shall not return. See also 15.5.2.~~

~~3 Default behavior: The implementation's default unexpected_handler calls std::terminate().~~

~~D.8.2 set_unexpected [set.unexpected]~~

~~unexpected_handler set_unexpected(unexpected_handler f) noexcept;~~

~~1 Effects: Establishes the function designated by f as the current unexpected_handler.~~

~~2 Remark: It is unspecified whether a null pointer value designates the default unexpected_handler.~~

~~3 Returns: The previous unexpected_handler.~~

~~D.8.3 get_unexpected [get.unexpected]~~

~~unexpected_handler get_unexpected() noexcept;~~

~~1 Returns: The current unexpected_handler. [Note: This may be a null pointer value. — end note]~~

~~D.8.4 unexpected [unexpected]~~

~~{[noreturn]} void unexpected();~~

~~1 Remarks: Called by the implementation when a function exits via an exception not allowed by its exception specification (15.5.2), in effect after evaluating the throw-expression (D.8.1). May also be called directly by the program.~~

~~2 Effects: Calls the current unexpected_handler function. [Note: A default unexpected_handler is always considered a callable~~

handler in this context. —*end note* }

Acknowledgements

VIII. References

- [N0741](#) Exceptions specifications in the Standard Library, Mats Henricson
- [N3051](#) Deprecating Exception Specifications, Doug Gregor
- [N4533](#) Make exception specifications be part of the type system, version 3, Jens Maurer
- [N4086](#) Removing trigraphs??!, Richard Smith
- [N4168](#) Removing `auto_ptr`, Billy Baker
- [N4190](#) Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- [P0001R0](#) Remove Deprecated Use of the `register` Keyword
- [P0002R0](#) Remove Deprecated `operator++(bool)`
- [P0004R0](#) Remove Deprecated `iostreams` aliases